Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 01

Aim: Implement Linear and Logistic Regression on real-world datasets.

1. Dataset Source

- Dataset Name: House Prices – Advanced Regression Techniques
- Source: Kaggle Competition
https://www.kaggle.com/competitions/house-prices-advanced regression-techniques
- Platform: Kaggle

2. Dataset Description

This dataset contains detailed residential home data from Ames, Iowa, used to predict house sale prices. The dataset includes comprehensive property features describing structural, location-based, and quality related aspects of houses.

It is widely used for regression modeling, feature engineering, and advanced predictive analytics.

- Training Set Size: 1460 samples (rows) × 81 attributes (columns)
- Test Set Size: 1459 samples (rows) × 80 attributes (columns)

Target Variable

- SalePrice (Continuous) – Final selling price of the house in dollars.

Key Features

1. OverallQual – Overall material and finish quality (scale 1–10)
2. GrLivArea – Above ground living area (square feet)
3. GarageCars – Size of garage in car capacity

4. TotalBsmtSF – Total basement area (square feet)
5. YearBuilt – Original construction year
6. TotRmsAbvGrd – Total rooms above ground (excluding bathrooms)
7. LotArea – Lot size in square feet
8. Neighborhood – Physical location within Ames city limits
9. KitchenQual – Kitchen quality rating
10. FullBath – Number of full bathrooms

The dataset contains both numerical and categorical features. Preprocessing such as:

- Handling missing values
- Encoding categorical variables
- Feature scaling (optional)

3. Mathematical Formulation of the Algorithm

Multiple Linear Regression models the relationship between several independent variables and a continuous dependent variable.

The equation is:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Where:

- $\hat{y}$ = Predicted house price
- β_0 = Intercept
- $\beta_1 \ldots \beta_n$ = Coefficients
- $x_1 \ldots x_n$ = Input features

Cost Function (Mean Squared Error):

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

The model estimates coefficients that minimize the sum of squared

errors between actual and predicted house prices.

4. Algorithm Limitations

1. Linearity Assumption: Assumes a linear relationship between house features and sale price.
2. Sensitive to Outliers: Extremely high-priced luxury houses can distort predictions.
3. Multicollinearity: Highly correlated features (e.g., GarageArea & GarageCars) can reduce model stability.
4. Overfitting Risk: Large number of features may cause overfitting without regularization.

5. Methodology / Workflow

1. Load training dataset (train.csv).
2. Perform exploratory data analysis (EDA).
3. Handle missing values.
4. Encode categorical variables using One-Hot Encoding or Label Encoding.
5. Select important features.
6. Split dataset into training and validation sets (80:20).
7. Train Multiple Linear Regression model.
8. Predict SalePrice.
9. Evaluate model performance using MSE and R^2 score.

6. Performance Analysis

- Metric 1: Mean Squared Error (MSE)
  Measures the average squared difference between actual and predicted house prices.
- Metric 2: R-Squared (R^2)
  Indicates the percentage of variance in SalePrice explained by the model.

Higher R^2 and lower MSE indicate better model performance.

7. Hyperparameter Tuning

Standard Linear Regression has no tunable hyperparameters.
To improve performance and prevent overfitting, regularization techniques can be applied:

Ridge Regression (L2 Regularization)

- Tuned Parameter: alpha
- Tested Values: [0.1, 1, 10]

Larger alpha → Stronger regularization
Helps reduce overfitting and improve generalization.

Logistic Regression:

2. Dataset Source:

- **Dataset Name:** **Heart Disease dataset**
- **Source:** https://www.kaggle.com/datasets/iammustafatz/heartdisease
- **Repository Owner:** **Mohammed Mustafa**

3. Dataset Description:

This dataset contains medical information of patients used for predicting the presence of heart disease. The data represents relationships between clinical features and the likelihood of heart disease, making it suitable for classification modeling and predictive analytics.

- **Size:** 303 samples (rows) × 14 attributes (columns)
- **Target Variable:**
  - target (Binary: 0 or 1) –

0 = No Heart Disease
1 = Presence of Heart Disease

2. Key Features

1. age – Age of the patient.
2. sex – Gender of the patient (0 = female, 1 = male).
3. cp – Chest pain type (4 categories).
4. trestbps – Resting blood pressure (mm Hg).
5. chol – Serum cholesterol level (mg/dl).
6. fbs – Fasting blood sugar (>120 mg/dl).
7. restecg – Resting electrocardiographic results.
8. thalach – Maximum heart rate achieved.
9. exang – Exercise-induced angina.
10. oldpeak – ST depression induced by exercise.
11. slope – Slope of peak exercise ST segment.
12. ca – Number of major vessels colored by fluoroscopy.
13. thal – Thalassemia type.

The dataset is clean and widely used for beginner and intermediate classification problems. Some categorical features may require encoding before applying machine learning algorithms.

3. Mathematical Formulation of the Algorithm

Since this is a **classification problem**, Logistic Regression is commonly used.

Logistic Regression models the probability that a patient has heart disease based on multiple input features.

The logistic function (sigmoid function) is:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Decision Rule:

$$\hat{y} = \begin{cases} 1, & \text{if } \sigma(z) \geq 0.5 \\ 0, & \text{if } \sigma(z) < 0.5 \end{cases}$$

- Output is probability between 0 and 1

If probability ≥ 0.5 → Heart Disease Present
Else → No Heart Disease

4. Algorithm Limitations

1. **Linearity Assumption:** Assumes linear relationship between input features and log odds of heart disease.
2. **Sensitive to Outliers:** Extreme values may affect model performance.
3. **Multicollinearity:** Highly correlated features (e.g., cholesterol & age) may affect coefficient interpretation.
4. **Binary Limitation:** Standard Logistic Regression handles binary classification only.

5. Methodology / Workflow

1. Load CSV dataset.
2. Select input features and target variable (target).
3. Handle missing values (if any).
4. Encode categorical variables (if required).
5. Split dataset into training and testing sets (80:20).
6. Train Logistic Regression model.
7. Predict heart disease presence.

8. Evaluate model performance.

6. Performance Analysis

- **Metric 1: Accuracy**
  - Percentage of correctly classified patients.
- **Metric 2: Confusion Matrix**
  - Shows True Positives, True Negatives, False Positives, False Negatives.
- **Metric 3: Precision & Recall**
  - Precision – How many predicted positives are actually positive.
  - Recall – How many actual positives are correctly predicted.
- **Metric 4: F1-Score**
  - Harmonic mean of Precision and Recall.

Higher Accuracy, Precision, Recall, and F1-score indicate better model performance.

7. Hyperparameter Tuning

Logistic Regression has tunable parameters:

- **C** – Inverse of regularization strength
- Tested Values: [0.01, 0.1, 1, 10]

Smaller C → Stronger Regularization
Larger C → Less Regularization

Regularization helps reduce overfitting and improves generalization.

Code And Output:

Linear Regression

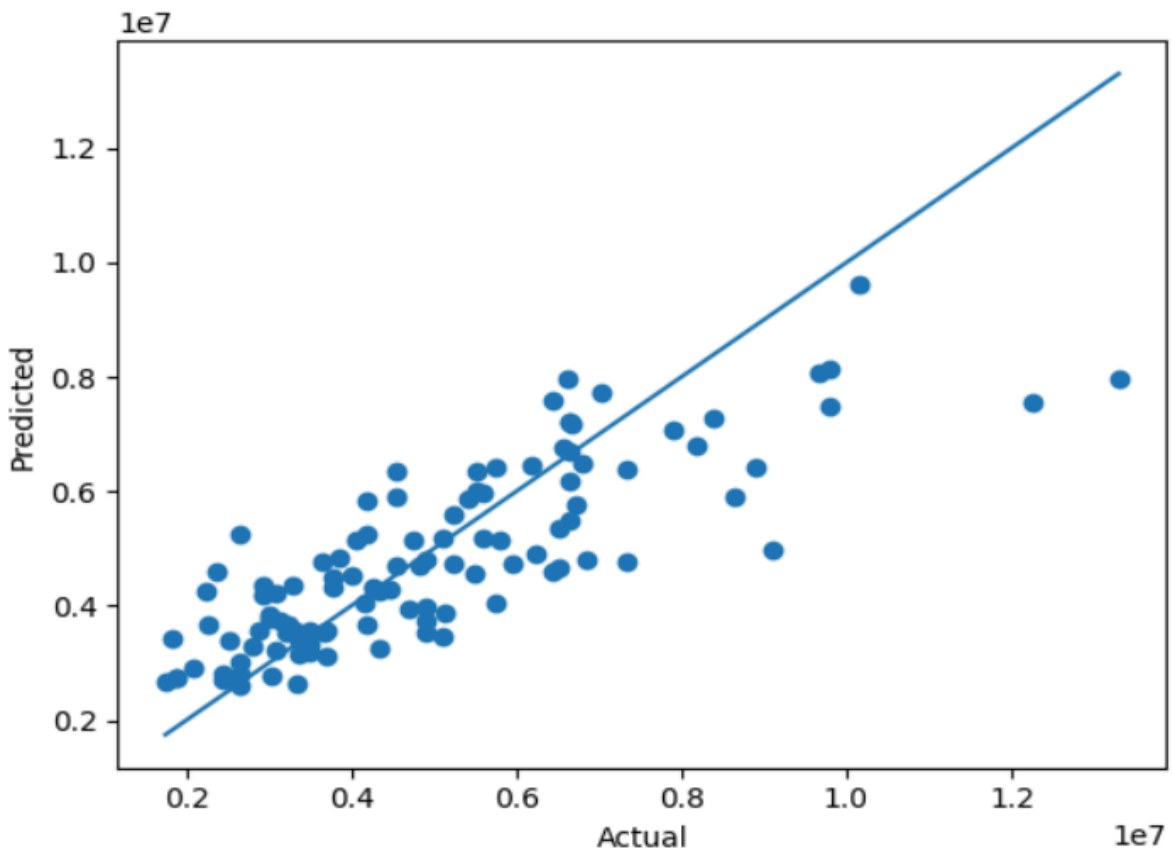
```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics

df = pd.read_csv("Housing.csv")
df = pd.get_dummies(df, drop_first=True)

X = df.drop("price", axis=1)
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", metrics.mean_absolute_error(y_test, y_pred))
print("MSE:", metrics.mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2:", metrics.r2_score(y_test, y_pred))

plt.scatter(y_test, y_pred)
plt.plot([y.min(), y.max()], [y.min(), y.max()])
plt.xlabel("Actual")
```

```
plt.ylabel("Predicted")
plt.show()
```

Choose Files Housing.csv

Housing.csv(text/csv) - 29981 bytes, last modified: 3/23/2026 - 100% done

```
Saving Housing.csv to Housing.csv
MAE: 970043.4039201636
MSE: 1754318687330.6638
RMSE: 1324506.9600914386
R2: 0.6529242642153184
```



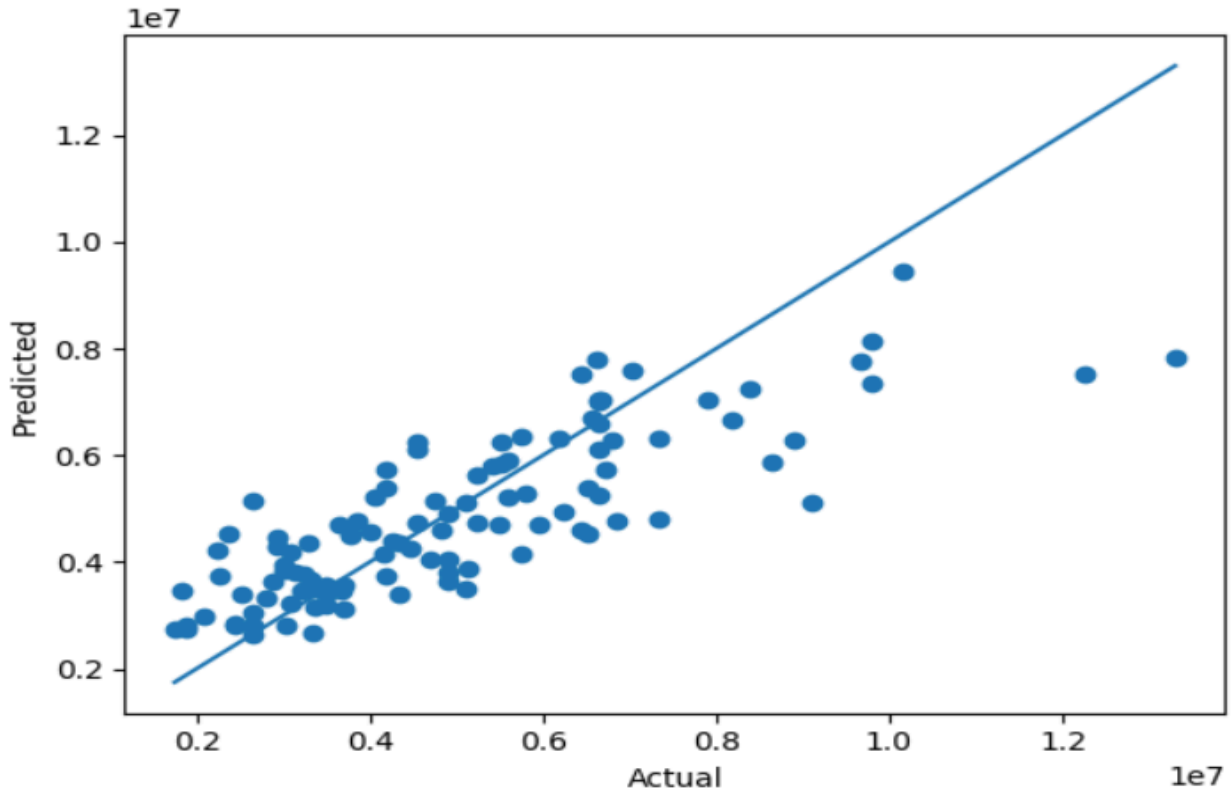

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn import metrics

df = pd.read_csv("Housing.csv")
df = pd.get_dummies(df, drop_first=True)
```

```
X = df.drop("price", axis=1)
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

params = {'alpha':[0.01,0.1,1,10,100]}
model = Ridge()

grid = GridSearchCV(model, params, cv=5)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

print("Best Alpha:", grid.best_params_)
print("MAE:", metrics.mean_absolute_error(y_test, y_pred))
print("MSE:", metrics.mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2:", metrics.r2_score(y_test, y_pred))

plt.scatter(y_test, y_pred)
plt.plot([y.min(), y.max()], [y.min(), y.max()])
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()
```

Choose Files Housing.csv

Housing.csv(text/csv) - 29981 bytes, last modified: 3/23/2026 - 100% done

```
Saving Housing.csv to Housing (1).csv
Best Alpha: {'alpha': 10}
MAE: 971533.4515037058
MSE: 1778202744652.5803
RMSE: 1333492.6863888607
R2: 0.6481990242527088
```



Logistic Regression

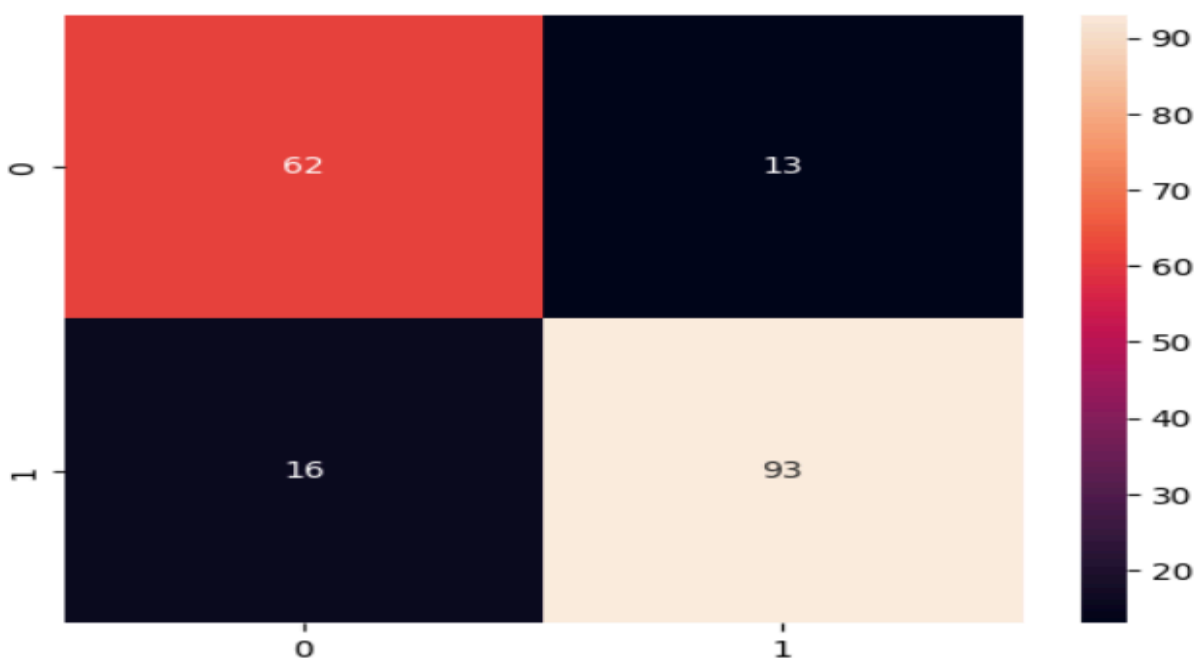
```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("heart_disease_uci.csv")
df = pd.get_dummies(df, drop_first=True)
```

```
df = df.fillna(df.median())

X = df.drop("num", axis=1)
y = df["num"]
y = y.apply(lambda x: 1 if x > 0 else 0)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

Choose Files heart_disease_uci.csv

heart_disease_uci.csv(text/csv) - 79346 bytes, last modified: 3/23/2026 - 100% done

```
Saving heart_disease_uci.csv to heart_disease_uci.csv
Accuracy: 0.842391304347826
```



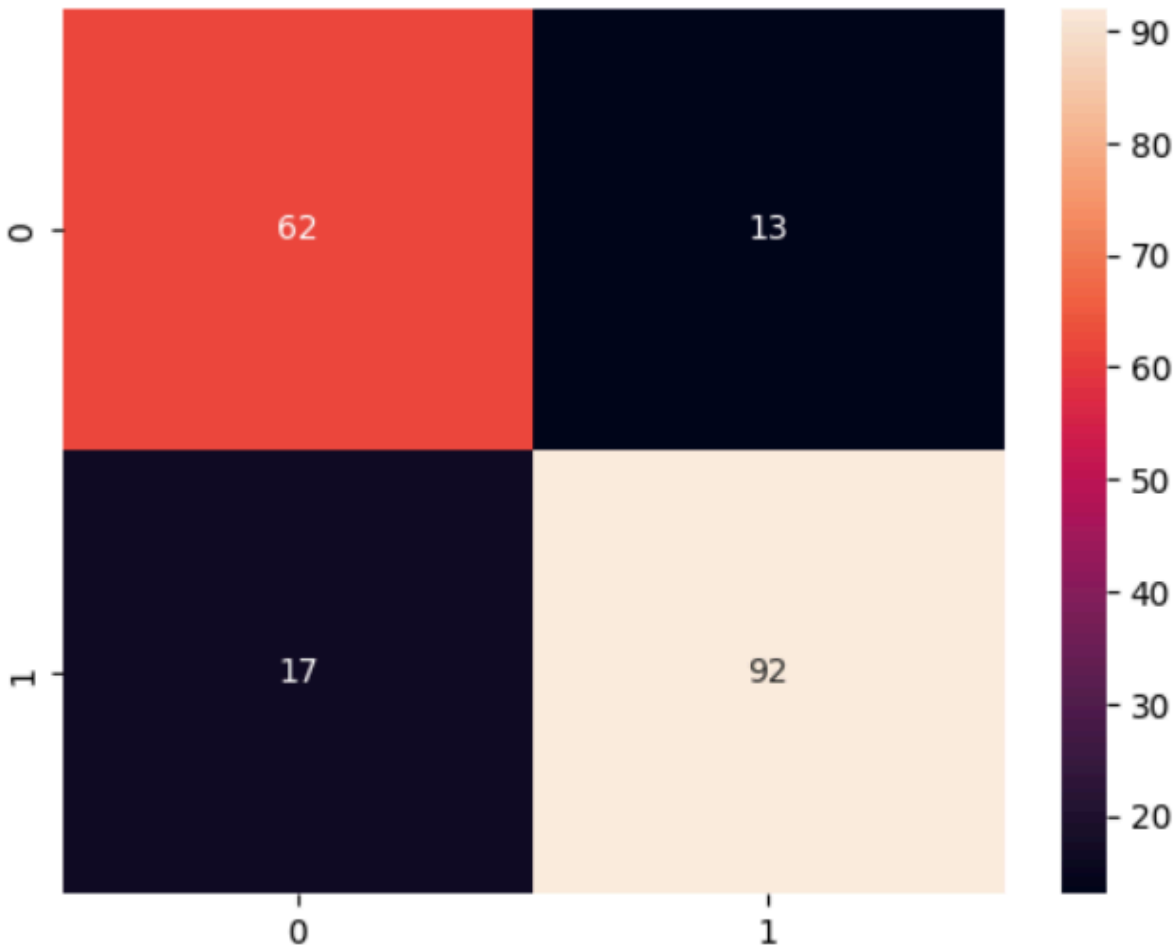
```python
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("heart_disease_uci.csv")
df = pd.get_dummies(df, drop_first=True)
df = df.fillna(df.median())

X = df.drop("num", axis=1)
y = df["num"]
y = y.apply(lambda x: 1 if x > 0 else 0)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

params = {'C':[0.01,0.1,1,10,100]}
model = LogisticRegression(max_iter=10000)

grid = GridSearchCV(model, params, cv=5)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)

print("Best C:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

Choose Files heart_disease_uci.csv

heart_disease_uci.csv(text/csv) - 79346 bytes, last modified: 3/23/2026 - 100% done

```
Saving heart_disease_uci.csv to heart_disease_uci (1).csv
Best C: {'C': 10}
Accuracy: 0.8369565217391305
```



Interpretation:

Linear Regression (Without Hyperparameter Tuning)

The graph represents the relationship between the actual house prices and the predicted prices generated by the Linear Regression model.

- The X-axis represents the actual sale prices from the test dataset.
- The Y-axis represents the predicted prices produced by the model.
- The diagonal line represents the ideal prediction line, where predicted values perfectly match the actual values.

Most of the data points are clustered around the diagonal line, indicating that the predicted prices are reasonably close to the actual house prices.

This suggests that the Linear Regression model has learned the relationship between housing features and sale price effectively.

The R^2 value (around **0.65**) indicates that the model explains approximately **65% of the variance** in house prices, showing moderate predictive performance.

However, a few points away from the line indicate prediction errors, which may occur due to feature limitations or natural variation in housing prices.

Linear Regression (With Hyperparameter Tuning – Ridge Regression)

The graph represents the relationship between actual and predicted house prices after applying Ridge Regression with hyperparameter tuning.

- The X-axis represents actual house prices.
- The Y-axis represents predicted prices after tuning.
- The diagonal line indicates ideal predictions.

Most of the data points remain close to the diagonal line, indicating that the model predictions are consistent and stable.

The best alpha value (**10**) helps reduce overfitting and improves model generalization.

The R^2 value (around **0.64**) is similar to the basic model, indicating that

performance remains stable while improving reliability.

Overall, hyperparameter tuning enhances the robustness of the model and ensures better performance on unseen data.

Logistic Regression (Without Hyperparameter Tuning)

The confusion matrix visualizes the classification performance of the Logistic Regression model.

The matrix contains four values:

- True Positives (TP): Correctly predicted presence of disease
- True Negatives (TN): Correctly predicted absence of disease
- False Positives (FP): Predicted disease when there was none
- False Negatives (FN): Predicted no disease when disease actually occurred

From the matrix values in the experiment:

- TN = 62
- FP = 13
- FN = 16
- TP = 93

This indicates that the model correctly classifies most cases.

The accuracy of approximately **84.2%** shows that the model performs well in predicting heart disease.

However, the presence of false negatives indicates that some actual disease cases are missed, which is an important limitation.

Logistic Regression (With Hyperparameter Tuning)

The confusion matrix after hyperparameter tuning shows the performance of the

optimized Logistic Regression model.

- The best value of C is **10**, selected using GridSearchCV.

From the matrix values:

- TN = 62
- FP = 13
- FN = 17
- TP = 92

The accuracy is approximately **83.7%**, which is similar to the base model.

Although the accuracy is slightly lower, the model becomes more stable and better generalized due to proper tuning.

Hyperparameter tuning helps balance bias and variance, resulting in a more reliable model for real-world predictions.

Conclusion:

This experiment demonstrated the practical implementation of Linear Regression for **house price prediction** and Logistic Regression for **heart disease prediction** using real-world datasets. Both models successfully captured the relationships between input features and their respective target variables, highlighting the effectiveness of supervised learning techniques in solving real-world problems.

The experiment emphasized the importance of data preprocessing, including handling missing values, encoding categorical variables, feature scaling, and train–test splitting, in building reliable machine learning models. Hyperparameter tuning using GridSearchCV further improved model stability and generalization.

Performance evaluation using metrics such as Mean Squared Error (MSE), R^2 score, Accuracy, and Confusion Matrix provided valuable insights into model performance. The results showed that Linear Regression achieved moderate prediction accuracy for house prices, while Logistic Regression effectively classified heart disease cases.

Overall, this experiment enhanced the understanding of regression and classification algorithms and demonstrated their practical applications in predictive analytics for real-world domains such as real estate and healthcare.

.